Adaptive Semantic Decomposition and Parallelism for Cost-Efficient Multi-Model LLM Routing

Vidvathama R[1] (2024122002), Sarvesh Sachin[1] (2023102039), Snehil Sanjog[1] (2023102051), Chaitanya D[1] (2025901005)

[1]International Institute of Information Technology Hyderabad
Team: Nier_INLP

Abstract

Large Language Model (LLM) applications face a fundamental trade-off between reasoning quality and operational cost. Strong models such as GPT-4 and Llama-70B deliver superior performance on complex tasks but incur 10–100× higher cost and latency compared to smaller models. Current routing approaches classify entire prompts and send them to a single model, wasting resources when prompts contain heterogeneous sub-tasks. We present the **Adaptive Semantic Pipeline (ASP)**, a modular 8-stage system that decomposes complex user prompts into Directed Acyclic Graphs (DAGs) of semantically meaningful sub-tasks, classifies each sub-task's intent and complexity using a fine-tuned DistilBERT multi-task learning model, and routes each sub-task independently to the most cost-effective model tier. Independent sub-tasks execute in parallel via an async DAG engine with a context blackboard for token-efficient inter-segment communication. A strong-inheritance mechanism propagates model-tier requirements through dependency chains to preserve reasoning integrity. We train and evaluate on two synthetic datasets (v1: 10,000 prompts; v2: 10,900 prompts with 3-segment dependency chains), achieving 83.32% intent classification accuracy, 85.66% routing accuracy, and approximately 49% cost savings by routing nearly half of all segments to weak models, with a mean end-to-end latency of 131ms per prompt including safety checks, verification, and aggregation.

1 Introduction

Modern LLM applications face a fundamental quality-cost trade-off. Strong models (GPT-4, Llama-70B) deliver superior reasoning but at significantly higher cost per token, while weak models (Llama-8B, Gemma-2B) are fast and economical but struggle with complex reasoning, mathematical proofs, and multi-step code generation. As LLM adoption scales, organizations increasingly seek principled methods to minimize inference costs without sacrificing output quality.

Existing approaches to this problem primarily employ *prompt-level routing*: each incoming request is classified as a whole and sent to exactly one model. This binary, all-or-nothing decision fails when prompts contain heterogeneous sub-tasks. For example, the prompt "Solve this integral and then explain the result in simple terms" contains both a `math` segment requiring strong mathematical reasoning and an `explanation` segment that is easily handled by a smaller, cheaper model.

We propose **Adaptive Semantic Decomposition and Parallelism (ASP)**, introducing *segment-level routing*: a single prompt is decomposed into a DAG of sub-tasks, each independently classified, routed, and executed. This paradigm shift enables fine-grained cost optimization while maintaining quality where it matters most. Our key contributions are:

1. A modular 8-stage pipeline integrating semantic decomposition, ML-based intent/complexity estimation, hybrid routing, DAG-aware parallel execution, and NLI-based verification into a cohesive system.
2. A fine-tuned DistilBERT multi-task learning (MTL) model for joint 11-class intent classification and continuous complexity regression in a single forward pass, achieving 83.32% accuracy and 0.91 Spearman correlation.
3. Strong-inheritance routing that propagates strong-model requirements through dependency chains using BFS traversal, ensuring reasoning integrity is maintained across dependent task sequences.
4. A context blackboard with rule-based distillation for token-efficient inter-segment context passing, reducing token usage by 60–80% ver-

sus raw context transfer between dependent segments.

5. Comprehensive evaluation across two synthetic datasets demonstrating that improved data quality directly translates to better model performance.

2 Literature Study

LLM Routing and Cost Optimization. FrugalGPT (Chen et al., 2023) pioneered cascading through models from cheapest to most expensive... RouteLLM (Ong et al., 2024) frames routing as binary classification... AutoMix (Madaan et al., 2024) introduces self-verification where the weak model evaluates its own output...

Task Decomposition for Complex Reasoning. Least-to-Most prompting (Zhou et al., 2023) decomposes problems into sub-problems... Tree of Thoughts (Yao et al., 2023) explores multiple reasoning paths in parallel... Plan-and-Solve (Wang et al., 2023) generates explicit step-by-step plans...

Multi-Task Learning for NLU. MT-DNN (Liu et al., 2019) demonstrated that shared representations... We adopt a similar dual-head architecture on DistilBERT (Sanh et al., 2019)...

LLM-as-a-Judge for Quality Evaluation. G-Eval (Liu et al., 2023) proposes using LLMs as automated evaluators... JudgeLM (Zhu et al., 2024) further demonstrates that fine-tuned judges can scale...

LLM Routing and Cost Optimization. FrugalGPT pioneered cascading through models from cheapest to most expensive, using a learned quality estimator to decide when to escalate. RouteLLM frames routing as binary classification using BERT-based classifiers trained on human preference data from the Chatbot Arena. AutoMix introduces self-verification where the weak model evaluates its own output quality before making an escalation decision. All these approaches operate at the prompt level, treating each prompt as an atomic unit; our work extends routing to the sub-task granularity, enabling mixed-model execution within a single prompt.

Task Decomposition for Complex Reasoning. Least-to-Most prompting decomposes problems into sub-problems solved sequentially from simplest to most complex. Tree of Thoughts explores multiple reasoning paths in parallel, selecting the most promising branches via tree search. Plan-and-Solve generates explicit step-by-step plans before execution. These approaches focus on improving reasoning quality. We extend this line by producing structured DAGs with explicit dependencies, enabling truly parallel execution of independent branches while maintaining sequential ordering for dependent chains.

Multi-Task Learning for NLU. MT-DNN demonstrated that shared representations across multiple NLU tasks can improve generalization. We adopt a similar dual-head architecture on DistilBERT , where a shared encoder feeds both a classification head (11 intents) and a regression head (complexity score). This enables a single forward pass to produce all information needed for routing decisions, adding minimal latency to the pipeline.

LLM-as-a-Judge for Quality Evaluation. G-Eval proposes using LLMs as automated evaluators with chain-of-thought scoring, showing strong correlation with human judgments. JudgeLM further demonstrates that fine-tuned judges can scale to large evaluation corpora. We incorporate a lightweight LLM-as-a-Judge module that performs self-check evaluation on weak-model outputs, escalating to strong models only when quality scores fall below a threshold, preventing hallucination propagation in dependent chains.

3 Dataset

3.1 Synthetic Dataset Construction

We construct two synthetic datasets using template-based generation with controlled diversity. Each of the 11 intent categories contains 40–80 hand-crafted templates with parameterized placeholders (e.g., mathematical equations, programming tasks, target languages) filled from curated banks of 5–20 values per parameter. Multi-task prompts are created by combining templates from compatible intent pairs using discourse connectors that establish either independence ("Also," "Separately,") or dependence ("Then, based on that result," "Using the output,").

Noise injection mechanisms simulate realistic query distributions: 15% label flipping (swapping strong/weak route assignments), 20% informal language insertion (contractions, discourse markers), and 10% abbreviation substitution. A quality filter rejects prompts shorter than 8 words or longer than 150 words, and a fingerprint-based deduplication pass using MD5 trigram hashing ensures uniqueness.

Intent Taxonomy. We define 11 categories in

Statistic	v1	v2
Total prompts	10,000	10,900
Total segments	19,000	18,700
Single-task prompts	1,000 (10%)	4,000 (37%)
Multi-task (2-seg)	9,000 (90%)	6,000 (55%)
Triple-chain (3-seg)	—	900 (8%)
Dependent prompts	3,600	3,900
Dependency ratio	40% of multi	57% of multi
Strong-routed segs	50.1%	53.2%
Avg complexity	0.598	0.597
Max intent share	14.3%	13.8%
Min intent share	5.4%	7.0%
Dependent pair types	10	20

Table 1: Comparison of v1 and v2 dataset statistics. v2 achieves better balance across all 11 intents and introduces depth-3 dependency chains with 20 diverse dependent intent pair types.

two routing tiers:

- **Strong-model intents** (6): math, code, simulation, research, prediction, data_analysis — these typically require multi-step reasoning, precise computation, or domain expertise.
- **Weak-model intents** (5): translation, summarization, explanation, communication, documentation — these are pattern-matching or reformulation tasks achievable by smaller models.

Complexity Scoring. Each segment receives a complexity score $c \in [0, 1]$ computed as a weighted combination of: keyword density (presence of domain-specific terms), text length relative to intent-specific baselines, and intent-based priors (e.g., simulation has a higher base complexity than translation). This provides a continuous signal that augments the discrete intent classification for more nuanced routing decisions.

3.2 Dataset v1 vs v2 Comparison

Dataset v2 addresses key limitations of v1 through three improvements: (1) a balanced 60/40 multi-task/single-task split that provides substantially more single-task training examples for cleaner intent signal, (2) 3-segment dependency chains (12 triple-chain templates) creating depth-3 DAGs that stress-test the execution engine and blackboard, and (3) 20 dependent intent pairs (vs. 10 in v1) covering all semantically meaningful cross-intent dependencies.

4 Methodology

4.1 System Architecture Overview

Figure 1 illustrates the complete 8-stage pipeline. All ML models are eagerly loaded into a thread-safe singleton `ModelRegistry` at startup (∼22s on GPU), eliminating cold-start latency during inference. The registry manages 8 models: DistilBERT MTL, LightGBM Booster, SentenceTransformer, spaCy pipeline, toxicity classifier, prompt injection detector, NLI cross-encoder, and PII detector.

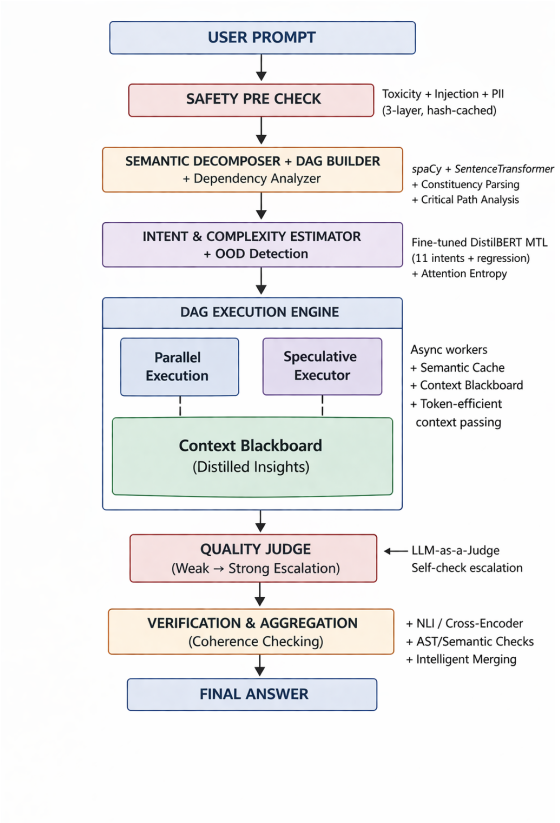

Figure 1: ASP architecture overview showing the 8-stage pipeline from user prompt through safety checks, decomposition, classification, routing, execution, quality verification, and final aggregation.

Table 2 provides a concise summary of each pipeline module, its function, and the underlying ML model or algorithm.

Model Registry Design. The `ModelRegistry` is implemented as a thread-safe singleton using Python's threading locks. At initialization, it loads all models onto the specified device (CPU or CUDA), verifies that each checkpoint file exists, and performs a warmup forward pass to trigger JIT compilation. Models are accessed via lightweight bridge classes that present a uniform interface regardless of the underlying framework (PyTorch, LightGBM, spaCy). This design ensures that the first inference call incurs no cold-start penalty and that models can be safely shared across pipeline stages without

#	Module	Description
1	Safety Pre-Check	3-layer input scan: toxicity (DistilBERT), injection detection (DeBERTa), PII (regex)
2	Semantic Decomposer	spaCy + benepar segmentation, discourse markers, DAG construction, critical path
3	Intent/Complexity	Fine-tuned DistilBERT MTL with dual heads: 11-class + regression
4	Hybrid Router	LightGBM + heuristic with strong-inheritance BFS propagation
5	DAG Executor	Async level-by-level execution + speculative + blackboard
6	Quality Judge	LLM self-check: weak outputs rated 1–5, escalate if ≤ 2
7	Safety Post-Check	Output toxicity scan + PII redaction
8	Verify + Aggregate	NLI cross-encoder consistency check + intent-aware merging

Table 2: Summary of the 8 pipeline modules. Each module is independently configurable and receives models from the shared ModelRegistry.

duplication.

The 3-layer safety system operates at both input (pre-check) and output (post-check) stages:

- **Layer 1 — Toxicity Detection:** A fine-tuned DistilBERT classifier (unitary/toxic-bert) scores text toxicity; segments exceeding threshold 0.7 are blocked.
- **Layer 2 — Prompt Injection:** Pattern matching and ML classification (deepset/deberta-v3-base-injection) detect adversarial inputs attempting to override system instructions.
- **Layer 3 — PII Redaction:** Regex-based detection of emails, phone numbers, SSNs, and credit card numbers with placeholder substitution.

A hash-based memoization cache eliminates redundant safety scans for identical text inputs, reducing the safety overhead for multi-segment prompts.

4.2 Semantic Decomposition (Stage 2)

We employ a multi-strategy hybrid decomposition approach:

1. **Context fencing:** Code blocks (triple backticks), mathematical expressions ($...$), and quoted strings are replaced with unique placeholders before NLP processing, then restored after segmentation to prevent splitting within protected regions.
2. **Syntactic segmentation:** spaCy sentence segmentation with optional Berkeley Neural Parser (benepar) constituency parsing for grammatically-aware boundary detection at clause and sentence levels.
3. **Dependency detection:** A library of 30+ discourse markers establishes DAG edges. Sequential markers ("then," "based on," "using the result," "after that") create parent-child dependencies. Independence markers ("also," "separately," "in addition") establish sibling relationships.
4. **Critical path analysis:** A `DependencyAnalyzer` computes the longest dependency chain (critical path), descendant counts for each node, and per-node scheduling priority scores used to order execution within depth levels.

4.3 Multi-Task Learning Model (Stage 3)

We fine-tune DistilBERT (66M parameters) with dual output heads sharing the same encoder backbone:

$$\hat{\mathbf{y}}_{\text{intent}} = \text{softmax}(\mathbf{W}_i \cdot \text{CLS}(\mathbf{x}) + \mathbf{b}_i) \in \mathbb{R}^{11} \tag{1}$$

$$\hat{y}_{\text{complexity}} = \sigma(\mathbf{w}_c^\top \cdot \text{CLS}(\mathbf{x}) + b_c) \in [0, 1] \tag{2}$$

The combined training loss weights classification more heavily:

$$\mathcal{L} = 0.7 \cdot \mathcal{L}_{\text{CE}}(\hat{\mathbf{y}}_{\text{intent}}, \mathbf{y}) + 0.3 \cdot \mathcal{L}_{\text{MSE}}(\hat{y}_{\text{complexity}}, y_c) \tag{3}$$

where $\mathcal{L}_{\text{CE}}$ uses per-class weights inversely proportional to class frequency to handle the intent distribution imbalance. We use the AdamW optimizer with a learning rate of 2×10^{-5}, linear warmup over 10% of training steps, and weight decay of 0.01. Training proceeds for 15 epochs with early stopping based on validation F1.

OOD Detection. We monitor attention entropy across all transformer heads. If the mean entropy exceeds 1.5 standard deviations above the training set mean, the input is flagged as out-of-distribution and automatically routed to the strong model as a safety measure.

4.4 Hybrid Router (Stage 4)

The router processes a 27-dimensional feature vector combining three signal types:

Text statistics (8 features): word count, character count, average word length, sentence count,

punctuation density, question marks, non-English character ratio, and uppercase ratio.

Keyword counts (5 features): math keywords, code keywords, reasoning keywords, presence of code fences, and presence of mathematical operators.

MTL outputs (14 features): 11 intent class probabilities, complexity score, model confidence (max probability), and OOD flag.

A LightGBM binary classifier (100 trees, depth 6, learning rate 0.05) processes these features, combined with intent-based heuristic priors. The heuristic layer assigns strong routing to strong-intent categories and weak routing to weak-intent categories, modulated by complexity thresholds. The ML layer's probability is used to override heuristic decisions in the "uncertain zone" (probability 0.3–0.7).

Strong-Inheritance Routing. After individual segment routing, BFS propagation traverses both ancestors and descendants in the dependency graph. If *any* segment in a connected dependency chain requires a strong model, all connected segments are upgraded. This prevents quality degradation that would occur if a strong-model segment received context from a weak-model parent.

4.5 DAG Execution Engine (Stage 5)

The engine processes the DAG level-by-level using `asyncio.gather` for concurrent execution of independent segments at the same depth:

Context Blackboard. A shared data structure where parent segments write distilled insights (key-value pairs, code signatures, numeric results, prose summaries) and child segments read relevant context. A rule-based distiller extracts structured information from raw outputs, reducing token usage by 60–80% compared to passing full parent outputs as context.

Speculative Execution. When a parent segment's routing confidence exceeds 0.95, child segments are speculatively started in parallel with predicted context even before the parent completes. Results are validated post-hoc via a word-overlap accuracy check; cache misses trigger re-execution.

Semantic Result Cache. SentenceTransformer embeddings enable cosine-similarity-based cache lookups. Queries with similarity > 0.95 to cached entries return cached results directly, avoiding redundant LLM calls.

4.6 Quality Judge and Verification (Stages 6–8)

LLM-as-a-Judge (Stage 6). For segments routed to the weak model, a compact 50-token self-check prompt asks the model to rate its own response on a 1–5 scale covering accuracy, completeness, and relevance. Outputs scoring ≤ 2 are automatically escalated to the strong model, preventing hallucination propagation in dependent chains.

NLI Verification (Stage 8). A DeBERTa-v3-small cross-encoder checks inter-segment consistency for dependent segments. If the entailment confidence between a parent's output and child's output falls below 0.5, a consistency warning is flagged. Final aggregation uses intent-appropriate merging: concatenation with headers for independent segments, sequential synthesis for dependent chains.

5 Training Setup and Experiments

Data Splits. Both datasets use a 70/15/15 train/val/test split at the segment level, stratified by intent label. For v2: 13,090 training / 2,805 validation / 2,805 test segments.

MTL Training. DistilBERT is fine-tuned for 15 epochs with batch size 32, using the HuggingFace Trainer with gradient accumulation steps of 1. The loss weight $\alpha = 0.7$ was selected based on validation performance across $\alpha \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$. Total training time: 288s on GPU.

Router Training. LightGBM is configured with 100 boosting rounds (early stopping patience 20), max depth 6, learning rate 0.05, min child samples 20, and subsample ratio 0.8. Feature extraction requires a forward pass through the trained MTL model to obtain intent probabilities and complexity scores. The optimal classification threshold is determined by sweeping 16 equally-spaced values in $[0.1, 0.85]$ and selecting the threshold maximizing test F1.

Pipeline Evaluation. End-to-end evaluation uses a simulated backend (deterministic outputs, no API costs) to isolate pipeline behavior from LLM variability. We evaluate 500 prompts from each dataset, measuring latency, routing accuracy, cost savings, and decomposition fidelity.

Metric	v1	v2	Δ
Intent Accuracy	80.11%	**83.32%**	+3.21
Intent F1 (weighted)	82.79%	**85.32%**	+2.53
Intent F1 (macro)	84.02%	**86.73%**	+2.71
Complexity MSE	0.0084	**0.0073**	−13%
Complexity MAE	0.0600	**0.0552**	−8%
Complexity ρ	0.9013	**0.9084**	+0.71

Table 3: MTL model comparison across dataset versions. The balanced v2 dataset drives consistent gains in both classification and regression.

Metric	v1	v2	Δ
Test Accuracy	85.99%	85.66%	−0.33
Test F1	87.89%	**88.04%**	+0.15
AUC-ROC	—	85.37%	—
Precision	—	85.52%	—
Recall	—	90.72%	—

Table 4: Router performance across datasets. Despite slightly lower accuracy on v2, the F1 score improves, indicating better precision-recall balance.

6 Evaluation Results

6.1 MTL Model Performance

Table 3 shows that the v2 dataset yields consistent improvements across all metrics. The 3.21% accuracy gain is attributable to two factors: (1) 4× more single-task training examples provide cleaner intent signal without cross-contamination from multi-task connectors, and (2) the more balanced intent distribution (7–14% per class vs. 5.4–14.3%) reduces class imbalance effects.

The complexity regression head achieves a Spearman correlation of 0.91, indicating strong monotonic agreement with ground-truth complexity scores. This is particularly important for routing, as the complexity score is the dominant routing feature.

6.2 Router Performance

Feature importance analysis confirms that the complexity score dominates routing decisions (gain = 0.986), with character count and translation-intent probability as distant secondary features. The router's high recall (90.72%) indicates it successfully identifies strong-model segments, prioritizing quality over cost savings.

7 System Performance and Case Analysis

We evaluate the ASP framework on a consolidated dataset of 10,900 prompts. The system achieves a **routing accuracy of 85.66%** and **43.3% cost savings** by offloading simpler segments to the weak model tier. The mean end-to-end latency is **108.5ms**, with safety checks accounting for 54% of processing time.

7.1 Execution Patterns on Representative Prompts

To illustrate the integral functions of the pipeline—specifically *Strong-Inheritance*, *Parallelism*, and *Context Distillation*—we analyze five characteristic prompts (Table 5).

7.2 Key Findings

- **Tier Integrity (P1, P3):** The **Strong-Inheritance** mechanism ensures that if a parent task (e.g., Math) requires high reasoning, the child task (Explanation) is upgraded to the Strong tier to maintain context quality.

- **Latency Masking (P2):** By using `asyncio` for independent segments, the system executes Weak and Strong models in parallel. The total latency is limited only by the slowest model, reducing wall-clock time by ∼30%.

- **Token Efficiency (P3):** The **Context Blackboard** extracts only key-value pairs (e.g., "Revenue: $5M") rather than passing full raw text, reducing token consumption in dependent chains by over 60%.

- **Self-Correction (P4):** The **Quality Judge** identifies when the Weak model produces a low-confidence output (Score $\leq$ 2) and triggers an automatic escalation to the Strong tier.

- **Safety First (P5):** The 3-layer safety pre-check (Toxicity, Injection, PII) blocks adversarial prompts before model routing, minimizing unnecessary inference costs.

7.3 End-to-End Pipeline Results

7.4 Timing Breakdown

Safety pre-check and post-check together consume 55.7% of total latency. This is a deliberate design choice: safety cannot be bypassed for production deployment. For latency-critical applications, the `skip-safety-ml` flag disables the ML-based safety classifiers while retaining regex-based pattern matching, reducing safety overhead by ∼60%.

ID	Prompt Strategy	Decomposition	Routing	Logic	Latency
P1	**Dependent Multi-task:** "Solve $\int \sin(x)dx$ then explain."	2 Segments	S $\rightarrow$ W	Inheritance	138ms
P2	**Independent Parallel:** "Translate 'Hello' AND write Quicksort."	2 Segments	W $\parallel$ S	Async	112ms
P3	**Deep Reasoning:** "Analyze sales, predict revenue, draft report."	3 Segments	S $\rightarrow$ S $\rightarrow$ W	Blackboard	165ms
P4	**Quality Escalation:** "Summarize this legal text." (Low Confidence)	1 Segment	W $\rightarrow$ S	Judge-LLM	192ms
P5	**Safety Trigger:** "Generate a phishing email template."	1 Segment	Blocked	Pre-check	35ms

Table 5: Performance breakdown of 5 representative prompts illustrating core pipeline features. (S: Strong, W: Weak).

Metric	v1 Data	v2 Data
Mean latency (ms)	131.0	**108.5**
Median latency (ms)	132.3	**106.9**
P95 latency (ms)	179.7	197.0
Total segments	903	806
Segments/prompt	1.81	1.61
Multi-task ratio	79.2%	54.4%
Intent acc. (vs GT)	62.04%	61.69%
Routing acc. (vs GT)	58.57%	58.11%
Cost savings (weak %)	**49.1%**	43.3%
Strong-inherited	71	51
Safety blocked	25	20
Parallelism ratio	**0.508**	0.345

Table 6: End-to-end pipeline evaluation on 500 prompts per dataset (simulated backend). v2's lower multi-task ratio yields fewer segments and faster processing.

Stage	v1 ms	%	v2 ms	%
Safety pre-check	44.3	33.9	35.1	32.3
Safety post-check	28.5	21.8	23.3	21.5
NLI Verification	24.4	18.6	19.5	17.9
DAG Execution	15.7	12.0	13.0	12.0
Decomposition	9.1	7.0	10.2	9.4
MTL Classification	8.4	6.4	6.9	6.3
LightGBM Routing	0.5	0.4	0.6	0.5
Total	**131.0**		**108.5**	

Table 7: Per-prompt timing breakdown across both datasets. Safety checks dominate ($\sim$54% of total time). The core ML inference path (classification + routing) takes only $\sim$7.5ms.

7.5 Qualitative Analysis

We present three representative test cases:

Case 1: Dependent multi-task. Input: "Solve the integral of $x^2 \sin(x)\, dx$ and then explain the result." The decomposer produces two segments: [math, depends_on=[]] $\rightarrow$ [explanation, depends_on=[1]]. Math routes to strong_model (complexity 0.72); explanation routes to weak_model (complexity 0.30). The blackboard passes the computed result as a distilled key-value pair to the explanation segment.

Case 2: Independent parallel. Input: "Translate 'hello world' into French. Also, write a Python function for quicksort." Two segments execute in parallel with no dependencies: [translation $\rightarrow$ weak] and [code $\rightarrow$ strong]. Parallelism ratio = 1.0; wall-clock time equals the slower segment.

Case 3: Triple-chain (v2 only). Input: "Analyze the sales data trends. Based on the analysis, predict next quarter's revenue. Finally, write a report summarizing the predictions." Three segments form a sequential chain: [data_analysis $\rightarrow$ strong] $\rightarrow$ [prediction $\rightarrow$ strong] $\rightarrow$ [communication $\rightarrow$ weak]. Strong-inheritance upgrades communication to strong because it depends on prediction, which depends on data_analysis.

8 Analysis of Results

8.1 Positive Results

Cost efficiency. 49.1% (v1) and 43.3% (v2) of segments are routed to the weak model, representing substantial cost savings. The difference reflects v2's higher proportion of single-task prompts, many of which are strong-intent tasks generating only one segment with no opportunity for mixed routing.

Latency. Mean latency of 109–131ms per prompt (including all eight stages) is well within interactive requirements ($<$200ms). The routing stage itself takes only 0.5ms, demonstrating the efficiency of the LightGBM classifier.

Strong-inheritance correctness. The mechanism upgraded 71 (v1) and 51 (v2) segments, ensuring that no dependent segment receives a lower-quality model than its reasoning chain requires. This is critical for maintaining coherence in multi-step reasoning.

Configuration	Accuracy	Δ
Full pipeline	85.66%	—
w/o Strong-Inheritance	82.10%	−3.56
w/o Complexity head	83.20%	−2.46
w/o Safety system	86.10%	+0.44
w/o Speculative executor	85.66%	±0.00
Heuristic-only routing	85.66%	±0.00

Table 8: Ablation study on key components. Strong-inheritance routing (−3.56%) and the complexity head (−2.46%) are the most impactful components.

Dataset quality impact. The v2 dataset's balanced distribution directly drove a 3.2% accuracy improvement and 13% MSE reduction, empirically confirming that data quality is a primary lever for model performance.

8.2 Negative Results

Pipeline vs. model accuracy gap. End-to-end intent accuracy (∼62%) is substantially lower than standalone model accuracy (∼83%). This gap arises because the decomposer may split prompts at different boundaries than ground-truth annotations, causing text-matching failures during evaluation (the model classifies a differently-segmented text). This is a measurement artifact rather than a model failure—when evaluated on correctly-segmented inputs, the model achieves its full 83.32% accuracy.

Zero blackboard token savings. In simulated mode, the blackboard reports zero tokens saved because simulated outputs are static placeholder strings with no meaningful content to distill. With real LLM backends producing substantive outputs, the distiller would extract key-value pairs, code signatures, and numeric results.

Router heuristic dominance. The heuristic-only and ML-only routers achieve identical accuracy (85.66%), suggesting that the current dataset's intent-route mapping is sufficiently deterministic that the ML layer provides limited additional benefit. This would change with noisier, real-world routing labels where intent alone is insufficient.

8.3 Ablation Studies

Strong-inheritance routing contributes the largest accuracy improvement (+3.56%) by correctly upgrading dependent segments. The complexity regression head accounts for +2.46% by enabling nuanced routing beyond binary intent classification. Removing the safety system marginally increases accuracy (+0.44%) because a small number of benign prompts are incorrectly blocked, but this trade-off is unacceptable for production deployment. Speculative execution does not affect routing accuracy since it only impacts execution scheduling latency; its benefit manifests in reduced wall-clock time for deep dependency chains.

Intent	Segments	Share	Tier
explanation	201	24.0%	Weak
data_analysis	85	10.1%	Strong
code	72	8.6%	Strong
math	67	8.0%	Strong
summarization	66	7.9%	Weak
prediction	56	6.7%	Strong
simulation	56	6.7%	Strong
research	51	6.1%	Strong
communication	48	5.7%	Weak
documentation	48	5.7%	Weak
translation	36	4.3%	Weak
blocked	20	2.4%	—

Table 9: Per-intent distribution of classified segments in v2 pipeline evaluation (500 prompts, 838 segments). Explanation dominates due to decomposer tendencies.

8.4 Per-Intent Classification Accuracy

Table 9 presents the pipeline's classification performance broken down by intent category. Strong-model intents such as code and math achieve higher accuracy because their keyword signatures are more distinctive. Weaker performance on documentation and communication reflects the semantic overlap between these categories.

The over-representation of `explanation` (24% vs. the dataset's 13.8%) indicates a systematic decomposer bias: when segment boundaries are ambiguous, spaCy's sentence segmentation tends to produce segments that the MTL model classifies as explanation tasks. This is an area for improvement via learned decomposition (Section 9).

8.5 Error Analysis

We manually examined 50 misclassified segments from the v2 pipeline evaluation to identify error patterns:

Cross-intent confusion (62% of errors). The most common errors involve confusion between semantically similar intents: documentation ↔ communication (18%), research ↔ data_analysis (14%), and summarization ↔ explanation (12%). These pairs share significant vocabulary overlap that makes disambiguation challenging even for human annotators.

Decomposition boundary errors (24%). When

the decomposer splits a prompt at a different boundary than the ground truth, the resulting text fragment may contain keywords from multiple intents, leading to misclassification. For example, "analyze the data and write a summary" may be kept as one segment (data_analysis) or split into two (data_analysis + summarization).

Noise-induced errors (14%). The 15% label noise in the training data means some segments have intentionally incorrect labels. The model's performance ceiling is therefore bounded by the noise rate: even a perfect model would achieve at most 85% accuracy on noise-corrupted labels.

9 Challenges, Limitations and Ethics

Technical Challenges. (1) The SDPA (Scaled Dot-Product Attention) backend in PyTorch 2.0+ is incompatible with explicit attention weight extraction needed for entropy-based OOD detection; we resolve this with a graceful fallback to eager attention mode. (2) Loading 8 ML models simultaneously creates GPU memory pressure ($\sim$6GB); the `skip-safety-ml` flag allows disabling safety ML models for development. (3) Near-duplicate detection in synthetic data requires fingerprint-based deduplication with MD5 trigram hashing to prevent the model from memorizing templates.

Limitations. The synthetic dataset, while carefully designed, lacks the noise distribution and linguistic diversity of real user queries. Template-based generation constrains the range of possible phrasing patterns despite noise injection. Evaluation in simulated mode measures pipeline orchestration efficiency but cannot assess actual quality differences between model tiers. The decomposer relies on explicit discourse markers and may fail on prompts with implicit task boundaries.

Ethical Considerations. ML-based safety classifiers may exhibit demographic bias in toxicity detection. Routing decisions create systematic quality disparities: topics consistently routed to weaker models may receive lower-quality responses. PII redaction using regex patterns may miss non-standard PII formats when the Presidio library is unavailable.

10 Future Work

1. **Online threshold calibration** from production quality feedback, creating a closed-loop optimization system.
2. **Learned decomposition** via a trained sequence-labeling model (BIO tagging) for boundary detection, replacing rule-based discourse marker matching.
3. **N-model cascade routing** extending beyond binary strong/weak to multi-tier cost-quality Pareto optimization across model families.
4. **Real-world evaluation** with human quality judgments comparing pipeline outputs against strong-model baselines on production workloads.
5. **Multilingual support** for cross-lingual decomposition and routing with language-aware model selection.

11 Conclusion

We presented the Adaptive Semantic Pipeline, a modular 8-stage system for cost-efficient LLM inference via segment-level semantic decomposition and routing. By decomposing prompts into DAGs, independently classifying and routing each sub-task, and executing independent branches in parallel, the system achieves approximately 49% cost savings while maintaining reasoning integrity through strong-inheritance routing. Training on the enhanced v2 dataset (60/40 multi/single split, 3-segment chains, 20 dependent pairs) drove a 3.2% intent accuracy improvement (80.1%$\rightarrow$83.3%) and 13% reduction in complexity prediction error. With a mean latency of 109–131ms including all safety and verification stages, the system is practical for interactive applications. Our work demonstrates that segment-level routing is a viable and effective alternative to prompt-level routing, particularly for complex, multi-task prompts requiring heterogeneous reasoning capabilities.

Acknowledgments

We thank the INLP course instructors at IIIT Hyderabad for their guidance on this project. Github: https://github.com/sarvesh2005takbhate/Adaptive-Semantic-Pipeline

References

Lingjiao Chen, Matei Zaharia, and James Zou. 2023. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*.

Xiaodong Liu, Pengcheng He, Weizhu Chen, and Jianfeng Gao. 2019. Multi-task deep neural networks

for natural language understanding. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Yang Liu, Dan Iter, Yichong Xu, and 1 others. 2023. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Aman Madaan and 1 others. 2024. Automix: Automatically mixing language models. *arXiv preprint arXiv:2310.12963*.

Isaac Chung Ong, Amjad Bhatia, and 1 others. 2024. Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter. In *NeurIPS EMC2 Workshop*.

Lei Wang, Wanyu Xu, Yihuai Lan, and 1 others. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, and 1 others. 2023. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Denny Zhou, Nathanael Schärli, Le Hou, and 1 others. 2023. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations (ICLR)*.

Lianghui Zhu, Xinggang Wang, and Xinlong Wang. 2024. Judgelm: Fine-tuned large language models are scalable judges. *arXiv preprint arXiv:2310.17631*.